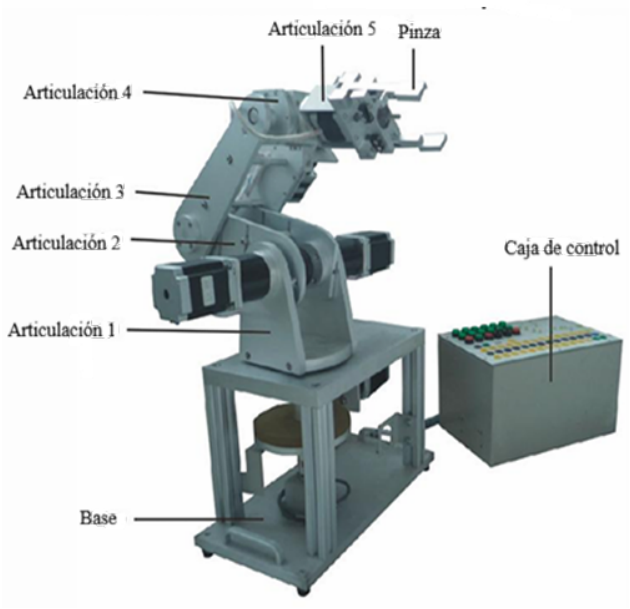
1. Descripción del Sistema: Brazo Robótico XK-LS5B

El brazo robótico de 6 GDL modelo XK-LS5B es un sistema antropomórfico utilizado con fines de entrenamiento. Su arquitectura se compone de una base y una cadena cinemática que permite el movimiento en el espacio tridimensional.

1.1. Componentes Principales



1.1.1. Estructura Mecánica

- **Base (Eje 1):** Permite la rotación horizontal del brazo completo. Incluye un motor NEMA 23 con transmisión por poleas.
- **Cadena Cinemática (Ejes 2-6):**
  - **Hombro (Eje 2):** Brazo grande, movido por motor NEMA 23 con acoplamiento directo.
  - **Codo (Eje 3):** Brazo pequeño, accionado por motor NEMA 23 con transmisión por correa síncrona.
  - **Muñeca (Eje 4):** Motor NEMA 17 para rotación de la pata.
  - **Giro de Muñeca (Eje 5):** Motor NEMA 17 con reductor de velocidad.
  - **Pinza (Eje 6):** Motor NEMA 17 para apertura/cierre de dedos.

1.1.2. Componentes Electrónicos

- **Controlador:** Microcontrolador PIC 16F1946 (8 bits, 14KB Flash, 54 pines E/S).
- **Drivers:** 2M542-N (para NEMA 23) y 2MA320 (para NEMA 17).
- **Sensores:**
  - Encoders incrementales (posición angular).
  - Sensores magnéticos NJK-5002C (fines de carrera, efecto Hall).
  - Finales de carrera mecánicos M102-011 (límites de pinza).
  - Potenciómetros (apertura de pinza).

1.2. Clasificación de Variables del Sistema

Cuadro 1: Clasificación de Variables del Brazo Robótico

Tipo	Símbolo	Definición	Componente Asociado
Variable Controlada (PV)	$y(t)$	Posición angular real de cada articulación $(\theta_1, \ldots, \theta_6)$	Encoder / Potenciómetro
Variable Manipulada (MV)	$u(t)$	Señales de control (PUL, DIR, ENA)	PIC 16F1946
Variable de Perturbación (DV)	$d(t)$	Peso del objeto, fricción, vibraciones, variación de voltaje	Carga externa, entorno
Referencia (Setpoint)	$r(t)$	Ángulos deseados para cada articulación	Operador / Programa
Error	$e(t)$	$e(t) = r(t) - y(t)$	Controlador (PIC)

2. Actividades Realizadas (ETN-1000)

2.1. Actividad 1: Diseño Conceptual de Telemetría (Arquitectura de 3 Capas)

Se ha diseñado un diagrama de arquitectura cliente-servidor de 3 capas que muestra el flujo de datos desde los sensores hasta la base de datos relacional. Este diagrama responde al requerimiento de la guía de laboratorio para la asignatura ETN-1000.

2.1.1. Estructura del Diagrama de 3 Capas

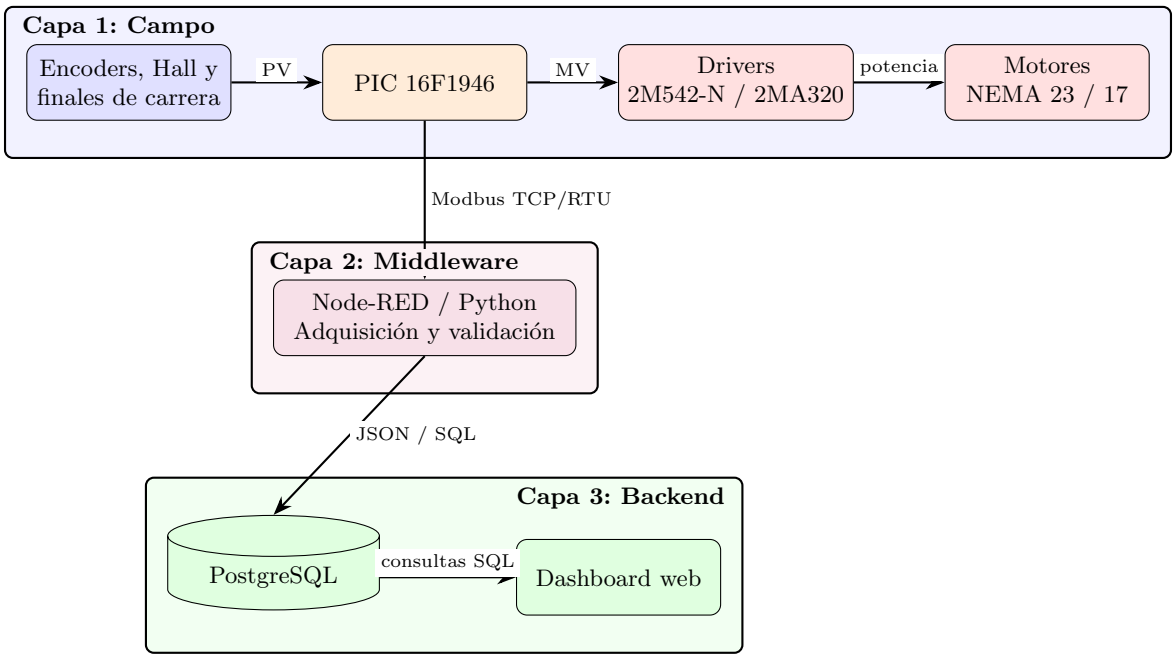


Figura 1: Arquitectura de 3 capas para el sistema de telemetría

2.1.2. Descripción del Flujo de Datos

1. **Capa de Campo:** Los sensores (encoders, sensores Hall, finales de carrera) miden la posición y los límites de las articulaciones. Estas señales analógicas o digitales son leídas por el microcontrolador PIC 16F1946, que también genera las señales de control (MV) para los motores a través de los drivers.

2. **Capa de Adquisición:** Un middleware (Node-RED o script Python) se comunica con el PIC mediante el protocolo Modbus TCP/RTU para leer los registros que contienen los valores de las variables. El middleware empaqueta esta información en formato JSON.

3. **Capa de Almacenamiento:** El middleware inserta los datos en la base de datos PostgreSQL, donde se almacenan de forma persistente. Un dashboard web consulta esta base de datos para mostrar en tiempo real los KPIs y el estado del sistema.

2.1.3. Análisis Detallado del Flujo de Datos desde el Sensor hasta la Base de Datos

El flujo de telemetría comienza en los sensores del brazo robótico. Los potenciómetros entregan señales analógicas relacionadas con la posición, mientras que los encoders incrementales y los finales de carrera producen señales digitales. La velocidad puede estimarse a partir de la variación de posición en el tiempo; en cambio, el par motor no se mide directamente sin un sensor de par o una estimación basada en la corriente del motor. Estas señales permiten conocer el estado de las articulaciones y verificar sus límites de operación.

1. Captura de Datos en el Nivel de Campo (Sensores) En esta etapa se adquieren las señales analógicas de los potenciómetros y las señales digitales de los encoders y finales de carrera. La Figura 2 resume el recorrido de estas mediciones hasta su almacenamiento.

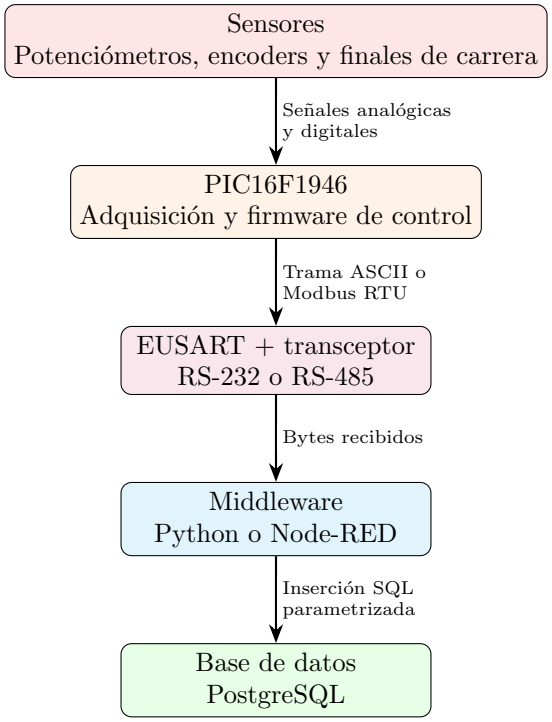


Figura 2: Flujo de datos desde los sensores hasta la base de datos

2. Acondicionamiento y Digitalización de Señales (PIC16F1946) Las señales llegan al sistema de control embebido basado en el microcontrolador **PIC16F1946**. Por ejemplo, los pines `RA0` y `RA1` pueden funcionar como los canales analógicos `AN0` y `AN1`, respectivamente; la asignación de cada canal a una articulación depende del cableado y debe documentarse en el diseño eléctrico.

El convertidor analógico-digital (ADC) del PIC16F1946 tiene una resolución de 10 bits y dispone de 17 canales. Su configuración utiliza los registros `ADCON0` y `ADCON1`; este dispositivo no posee un registro `ADCON2`. El resultado se almacena en `ADRESH` y `ADRESL`. Antes de cada conversión también deben configurarse el pin como entrada, su función analógica, las referencias de voltaje, el reloj del ADC y el tiempo de adquisición [**?**].

```
void ADC_Init(void) {
    TRISAbits.TRISA0 = 1;  // RA0 como entrada
    ANSELAbits.ANSA0 = 1;  // RA0 habilitado como AN0
    ADCON1 = 0b11110000;   // Resultado a la derecha, FRC, VDD/VSS
}

uint16_t ADC_Read(uint8_t channel) {
    ADCON0 = ((channel & 0x1F) << 2) | 0x01;
    __delay_us(10);
    ADCON0bits.GO_nDONE = 1;
    while (ADCON0bits.GO_nDONE) { }
    return ((uint16_t) ADRESH << 8) | ADRESL;
}
```

Listing 1: Ejemplo ilustrativo de lectura ADC con XC8

El ejemplo es orientativo: los nombres exactos de los campos de bits deben verificarse en el archivo de cabecera de la versión de MPLAB XC8 utilizada.

3. Procesamiento y Control en Tiempo Real Después de adquirir las señales, el firmware convierte las cuentas del ADC o del encoder a unidades físicas mediante los parámetros de calibración. A continuación, compara la posición medida con la referencia y comprueba los límites de seguridad antes de actualizar las salidas de control.

Para los drivers de motores paso a paso **2M542-N** y **2MA320**, las órdenes principales son pulsos de paso (PUL), dirección (DIR) y habilitación (ENA). La frecuencia de los pulsos determina la velocidad ordenada y su cantidad determina el desplazamiento. Por tanto, no debe describirse esta interfaz simplemente como una señal PWM, aunque ambas técnicas empleen trenes de pulsos. Los drivers proporcionan la etapa de potencia necesaria para energizar las bobinas de los motores.

Este flujo, desde la entrada de los sensores hasta la salida hacia los drivers, constituye el **ciclo de control en tiempo real** que gobierna los movimientos del brazo robótico.

4. Empaquetado y Transmisión Serial (EUSART) Paralelamente al control, el PIC16F1946 comunica el estado del brazo al nivel de supervisión. El firmware puede implementar Modbus RTU o una trama ASCII propia. Para mantener sencillo el ejemplo siguiente se utiliza una trama ASCII con identificador de articulación, sensor, variable de proceso, variable manipulada y estado.

Esta trama, que podría tener un formato como:

```
"A:1,S:101,PV:234,MV:1,EST:MOVIMIENTO\n"
```

Listing 2: Ejemplo de Trama Serial

La EUSART genera señales seriales de nivel lógico en sus pines `TX` y `RX`. Para conectarla físicamente a un enlace RS-232 o RS-485 se requiere un transceptor externo adecuado. El PIC16F1946 incorpora dos módulos EUSART compatibles con estos tipos de enlace [**?**].

```
void UART_SendString(const char *text) {
    while (*text != '\0') {
        while (!TXSTAbits.TRMT) { }
        TXREG = *text++;
    }
}

void SendTelemetry(void) {
    char buffer[72];
    snprintf(buffer, sizeof(buffer),
             "A:%u,S:%u,PV:%u,MV:%u,EST:%s\n",
             id_articulacion, id_sensor, valor_pv,
```

```
13                 valor_mv, estado);
14     UART_SendString(buffer);
15 }
```

Listing 3: Ejemplo ilustrativo de envío serial

5. Recepción, Procesamiento y Almacenamiento (Middleware + Base de Datos) El middleware recibe la trama, comprueba que tenga el formato esperado, convierte sus campos al tipo de dato correspondiente y ejecuta una inserción parametrizada. El siguiente ejemplo coincide con las columnas definidas posteriormente para la tabla `Telemetria`; en una implementación real también deben incorporarse reconexión, registro de errores y validación de rangos.

El controlador de PostgreSQL envía la consulta y sus parámetros por separado, lo que evita construir sentencias SQL mediante concatenación de texto. La base de datos conserva la marca de tiempo, la articulación, el sensor, los valores PV y MV y el estado operativo. Estos registros pueden emplearse para reconstruir trayectorias, analizar errores de seguimiento y generar reportes de operación o mantenimiento.

6. Resumen del Ciclo Completo del Dato En síntesis, las variables medidas se adquieren, convierten, validan, transmiten y almacenan junto con su contexto. La trazabilidad del dato exige conservar al menos la marca de tiempo, el origen de la medición, su unidad, el valor aplicado por el controlador y el estado del sistema.

Cuadro 2: Resumen del Flujo de Datos por Etapas

Etapa	**Origen**	**Formato de Datos**	**Destino**
1. Captura	Sensores	Señales analógicas o digitales	Entradas del PIC16F1946
2. Conversión	ADC / contadores	Valor digital de 10 bits o conteo	Firmware del PIC
3. Control	Firmware	Señales PUL, DIR y ENA	Drivers de motores
4. Empaquetado	Firmware / EUSART	Trama ASCII o Modbus RTU	Transceptor serial
5. Transmisión	Enlace RS-232/RS-485	Flujo de bytes	Middleware
6. Validación	Middleware	Datos tipados y validados	Consulta parametrizada
7. Persistencia	Middleware	Registro relacional	PostgreSQL

2.2. Actividad 2: Levantamiento de Requerimientos de Persistencia (Modelo Entidad-Relación)

Para cumplir con el segundo requerimiento de la guía, se realizaron entrevistas con los compañeros de las asignaturas de control (ETN-902 y ETN-1034) para identificar los datos críticos que deben ser almacenados. Como resultado, se propone el siguiente Modelo Entidad-Relación (MER) preliminar.

2.2.1. Entidades Identificadas y sus Atributos

Articulacion Representa cada uno de los 6 grados de libertad del brazo robótico.

- `id_articulacion` (PK): Identificador único (1-6).
- `nombre`: Nombre de la articulación (Base, Hombro, Codo, Muñeca, Giro Muñeca, Pinza).
- `tipo_motor`: Modelo del motor (NEMA 23, NEMA 17, NEMA 16).

- `limite_superior`: Ángulo máximo permitido (en grados).
- `limite_inferior`: Ángulo mínimo permitido (en grados).
- `relacion_transmision`: Relación de engranajes o poleas.

Sensor Representa cada uno de los sensores instalados en el sistema.

- `id_sensor` (PK): Identificador único.
- `id_articulacion` (FK): Relación con la articulación donde está instalado.
- `tipo`: Tipo de sensor (Encoder, Hall, Final de carrera, Potenciómetro).
- `modelo`: Modelo específico (NJK-5002C, M102-011).
- `unidad_medida`: Unidad de la variable que mide (grados, mm, booleano).
- `rango_min`: Rango mínimo de medición.
- `rango_max`: Rango máximo de medición.

Telemetria Registro histórico de las mediciones realizadas por los sensores.

- `id_registro` (PK): Identificador único del registro.
- `timestamp`: Marca de tiempo de la medición.
- `id_articulacion` (FK): Articulación a la que pertenece la medición.
- `id_sensor` (FK): Sensor que realizó la medición.
- `valor_pv`: Valor de la Variable de Proceso (PV) medida.
- `valor_mv`: Valor de la Variable Manipulada (MV) aplicada.
- `estado`: Estado del sistema en ese momento (En movimiento, Detenido, Error).

Alerta Registro de eventos anómalos o de seguridad.

- `id_alerta` (PK): Identificador único.
- `timestamp`: Momento en que se generó la alerta.
- `id_articulacion` (FK): Articulación donde ocurrió el evento.
- `tipo_alerta`: Tipo de alerta (Límite superior, Límite inferior, Sobrecalentamiento, Pérdida de pasos).
- `descripcion`: Descripción detallada del evento.
- `nivel_critico`: Nivel de criticidad (Bajo, Medio, Alto).

Tarea Registro de las tareas ejecutadas por el brazo robótico.

- `id_tarea` (PK): Identificador único.
- `nombre`: Nombre o descripción de la tarea.
- `timestamp_inicio`: Marca de tiempo de inicio.
- `timestamp_fin`: Marca de tiempo de finalización.
- `estado_tarea`: Estado de la tarea (Completada, Fallida, En progreso).

2.2.2. Diagrama Entidad-Relación

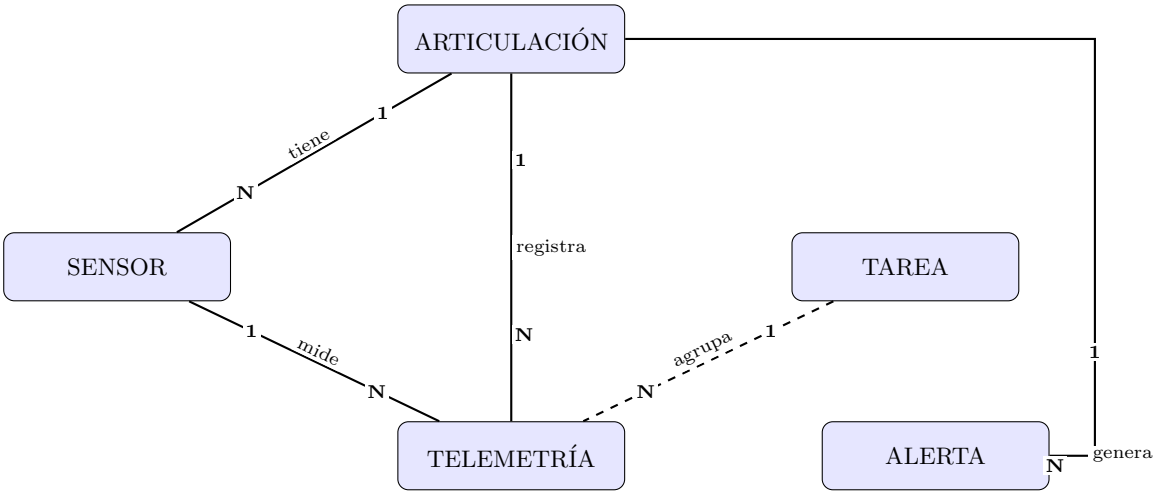


Figura 3: Modelo entidad-relación para el sistema de telemetría

3. Resultados y Discusión

3.1. Análisis de la Arquitectura Propuesta

La arquitectura de 3 capas diseñada cumple con los requerimientos de la guía, estableciendo un flujo de datos claro y escalable. La separación de capas permite:

- **Mantenibilidad:** Cada capa puede ser modificada o actualizada de forma independiente.
- **Escalabilidad:** El sistema puede crecer agregando más sensores o plantas sin afectar la lógica central.
- **Seguridad:** La base de datos puede implementar controles de acceso basados en roles (RBAC) como se menciona en la Fase 3 de la guía.

3.2. Justificación del Modelo Entidad-Relación

El MER propuesto cubre los requerimientos de persistencia indicados en la guía:

- **Variables de Proceso (PV):** Almacenadas en la tabla `Telemetria` a través del campo `valor_pv`.
- **Acciones de Control (MV):** Almacenadas en la tabla `Telemetria` a través del campo `valor_mv`.
- **Alertas de desborde:** Almacenadas en la tabla `Alerta`.
- **Tiempos de operación:** Almacenados en la tabla `Tarea`.

Además, la estructura permite realizar consultas complejas como:

- Histórico de posiciones de una articulación específica.
- Correlación entre acciones de control y respuestas del sistema.
- Análisis de eventos críticos y su frecuencia.

3.3. Consideraciones para la Implementación

Para las fases posteriores del laboratorio, se deben considerar los siguientes aspectos:

1. **Índices:** Crear índices en los campos `timestamp` para acelerar búsquedas temporales.
2. **Triggers:** Implementar triggers en PL/pgSQL para validar la integridad de los datos (ej. evitar valores nulos o fuera de rango).
3. **Conexión Modbus:** Configurar el middleware para leer los registros del PIC/PLC mediante Modbus TCP/RTU.
4. **Dashboard:** Desplegar una aplicación web que consulte la base de datos y muestre los KPIs en tiempo real.

4. Conclusiones

Se ha cumplido satisfactoriamente con los objetivos de la primera fase del laboratorio desde la perspectiva de ETN-1000 (Bases de Datos):

- Se diseñó una arquitectura de 3 capas que modela el flujo completo de datos desde los sensores hasta el dashboard de visualización, cumpliendo con el requerimiento de la guía.
- Se identificaron y clasificaron las variables críticas del sistema (PV, MV, DV) para el brazo robótico de 6 GDL.
- Se propuso un Modelo Entidad-Relación preliminar que cubre los requerimientos de persistencia, incluyendo telemetría, alertas y tareas.
- Se sentaron las bases para la implementación futura del middleware y la base de datos en PostgreSQL.

Este trabajo permite comprender la importancia del diseño de datos en sistemas de control industrial, demostrando cómo una arquitectura bien estructurada facilita la adquisición, almacenamiento y análisis de información para la toma de decisiones en tiempo real.

5. Recomendaciones y Trabajo Futuro

1. **Optimización de Consultas:** En la Fase 2, se recomienda crear índices en los campos `timestamp` y `id_articulacion` para optimizar las consultas de telemetría.
2. **Implementación de Triggers:** Desarrollar triggers que validen la integridad de los datos, como se especifica en la guía, para evitar el ingreso de valores nulos o fuera de rango.
3. **Documentación de la API:** Definir claramente la interfaz entre el middleware y la base de datos para facilitar el desarrollo colaborativo.
4. **Seguridad:** Implementar un sistema de control de acceso basado en roles (RBAC) para proteger los datos sensibles del sistema.
5. **Pruebas con Datos Sintéticos:** Generar datos de prueba para validar la estructura de la base de datos antes de la conexión con el hardware real.

Referencias

[1] Universidad Mayor de San Andrés. (2026). *Guía de Laboratorio Nº 1: ETN-902, ETN-1034, ETN-1000*.

[2] PostgreSQL Global Development Group. (2026). *PostgreSQL Documentation*. Recuperado de https://www.postgresql.org/docs/

[3] Modbus Organization. (2026). *Modbus Protocol Specification*. Recuperado de https://modbus.org/

[4] Node-RED. (2026). *Node-RED Documentation*. Recuperado de https://nodered.org/docs/